Access Modifiers في #C

تاسك سيرش: الفرق بين `protected internal` و `private protected`

1. الأول كده نفتكر الأساسيات

في #C كل حاجة بنكتبها (كلاس، ميثود، متغير) بنحدد مين يقدر "يشوفها" ويستخدمها، وده اسمه **Access Modifier**.
عشان نفهم الموضوع ده كويس لازم نفهم كلمتين:

- **Assembly:** يعني لو عندك Solution ببساطة هو المشروع بتاعك بعد ما يتعمله Build (ملف `.exe` أو `.dll`). فيه 2 مشروع، يبقى عندك 2 Assembly.

- **Inheritance** (**الوراثة**): إن كلاس يرث من كلاس تاني بـ `: Parent`.

والموديفايرز الأساسية اللي احنا عارفينها:

الموديفاير	بيتشاف فين؟
`public`	من أي مكان
`private`	جوه نفس الكلاس بس
`protected`	جوه الكلاس نفسه + الكلاسات اللي بترث منه (حتى لو في مشروع تاني)
`internal`	أي حد جوه نفس المشروع (Assembly)، ومحدش من بره يشوفه

طب المشكلة فين؟ المشكلة إن `protected` بيبص على **الوراثة** بس، و `internal` بيبص على **المشروع** بس. إحنا ساعات عايزين نجمع الاتنين مع بعض، وهنا بيدخلوا بطلين قصتنا. 😎

2. الأول: `protected internal`

يعني إيه؟

ده معناه "**protected أو internal**" (يعني **OR**). يعني تقدر توصله لو تحققت **أي حاجة** من الاتنين:

- أنت جوه نفس المشروع (Assembly) **حتى لو مش بترث**، أو
- أنت كلاس بترث من الكلاس ده **حتى لو في مشروع تاني**.

يعني ده أوسع واحد فيهم، وبيفتح الباب لناس أكتر.

مثال من الحياة: تخيل شركة، والـ Assembly هو الشركة، والوراثة هي إنك من العيلة. الباب ده بيدخل منه **أي موظف في الشركة** (حتى لو غريب)، **وكمان أي حد من العيلة** حتى لو شغال في شركة تانية. يعني "اللي من الشركة أو من العيلة يدخل".

3. التاني: `private protected`

يعني إيه؟

ده معناه "**protected و internal**" (يعني **AND**). لازم تحقق **الشرطين مع بعض**:

- تكون كلاس بيرث من الكلاس ده، **و**
- تكون في نفس المشروع (Assembly).

فيه دليل إنه "خاص" `private` بكتير. وكلمة `protected internal` يعني ده أضيق واحد فيهم، وده بيبقى مقفول أكتر من أكتر، يعني محصور في نطاق صغير.

مثال من الحياة: الباب ده مبيدخلش منه غير **واحد من العيلة وفي نفس الوقت شغال في نفس الشركة**. لو من العيلة بس وشغال بره؟ ما تدخلش. لو موظف في الشركة بس مش من العيلة؟ برضه ما تدخلش.

ملحوظة: `private protected` نزل في إصدار **C# 7.2**، فلو شغال على نسخة أقدم مش هتلاقيه.

4. جدول المقارنة (أهم حتة في الملف)

مين بيحاول يوصل؟	`protected internal`	`private protected`
نفس الكلاس	✔	✔
كلاس بيرث منه، في **نفس** المشروع	✔	✔
كلاس عادي (مش بيرث)، في **نفس** المشروع	✔	✘
كلاس بيرث منه، في مشروع **تاني**	✔	✘
كلاس عادي (مش بيرث)، في مشروع **تاني**	✘	✘

الخلاصة في جملة: الفرق كله في حرف الربط.

protected = `protected internal` **أو** internal (واسع).

protected = `private protected` **و** internal (ضيق).

5. مثال بالكود

تخيل عندنا مشروعين: **ProjectA** (ده اللي فيه الكلاس الأب) و **ProjectB** (مشروع تاني بيضيف ProjectA كـ Reference).

```
// ==================== ProjectA ====================
public class Parent
{
    protected internal string X = "protected internal";
    private protected  string Y = "private protected";
}

// كلاس عادي في نفس المشروع (مش بيرث)
class SameProjectNormal
{
    void Test()
    {
        Parent p = new Parent();
        Console.WriteLine(p.X);   // OK ✔ (لأنه في نفس المشروع)
        Console.WriteLine(p.Y);   // Error ✘ (مش بيرث)
    }
}

// كلاس بيرث في نفس المشروع
class SameProjectChild : Parent
{
    void Test()
    {
        Console.WriteLine(X);     // OK ✔
        Console.WriteLine(Y);     // OK ✔ (بيرث + نفس المشروع)
    }
}

// ==================== ProjectB ====================
// كلاس بيرث بس في مشروع تاني
class OtherProjectChild : Parent
{
    void Test()
    {
        Console.WriteLine(X);     // OK ✔ (لأنه بيرث)
        Console.WriteLine(Y);     // Error ✘ (مش في نفس المشروع)
    }
}

// كلاس عادي في مشروع تاني
class OtherProjectNormal
{
    void Test()
```

```
    {
        Parent p = new Parent();
        Console.WriteLine(p.X);   // Error ✘
        Console.WriteLine(p.Y);   // Error ✘
    }
}
```

6. نستخدم أنهي واحد وإمتى؟

`protected internal`

استخدمه لما تعمل مكتبة (Library) وعايز كلاسات المشروع بتاعك كلها تستخدم الميثود دي، **وكمان** المبرمجين اللي هيستخدموا المكتبة بتاعتك ويورثوا من الكلاس يقدروا يستخدموها ويعملولها Override.

`private protected`

استخدمه لما تكون عايز الحاجة دي تفضل "جوه البيت". يعني بس الكلاسات اللي بترث جوه مشروعك هي اللي تشوفها، ومحدش من بره المشروع يلمسها نهائي حتى لو ورث. ده بيحميك من إن حد يورث كلاسك ويبوّظ الـ logic الداخلي.

7. نقط لازم تاخد بالك منها

- الترتيب مش فارق: `protected internal` هي هي `internal protected`. وبرضه `private protected` هي `protected private`.
- لو مكتبتش أي موديفاير على الكلاس نفسه، الافتراضي بيبقى `internal`، ولو على الـ members (ميثودز ومتغيرات) بيبقى `private`.
- الموديفايرز دي بتتحط على الـ members (حقول، ميثودز، properties)، لكن **مينفعش** تحطهم على كلاس رئيسي (top-level). الكلاس الرئيسي مسموح له بس `public` أو `internal`.
- دول سؤال متكرر جداً في الإنترفيوهات، فاحفظ جدول المقارنة كويس. ☺

8. ملخص سريع

الموديفاير	الربط	الاتساع	بالعامية
`protected internal`	OR (أو)	واسع	أي حد من المشروع **أو** أي وريث
`private protected`	AND (و)	ضيق	وريث **وكمان** من نفس المشروع